School of Computer Science Engineering and Technology

Course – B.Tech.
Course Code – CSET-225
Year - 2026

Semester – 5th
Course Name - IMDAI
Semester - ODD

Max. Marks: 4

LAB ASSIGNMENT # 04

Implementing U-Net for Image Segmentation

Objective

The objective of this lab assignment is to understand and implement the U-Net architecture for image segmentation. Students will begin with a simple U-Net demonstration on a single image to learn the workflow of segmentation. Then, they will apply the U-Net model on the Oxford-IIIT Pet Dataset to perform full-scale semantic segmentation.

Dataset

1. **Single Image Demonstration**: Use any sample image (e.g., from local directory) to test the forward pass of U-Net.
2. **Oxford-IIIT Pet Dataset**: This dataset consists of pet images with corresponding pixel-level annotations. The task is to train U-Net for semantic segmentation of pets.

Tasks

1. Load a single image, preprocess it, and pass it through a small U-Net model. Visualize input and output masks.
2. Understand U-Net architecture: encoder (downsampling), bottleneck, and decoder (upsampling with skip connections).
3. Load Oxford-IIIT Pet dataset (images + segmentation masks). Preprocess: resize, normalize, and prepare masks.
4. Implement U-Net in TensorFlow/Keras with convolution, pooling, upsampling, and skip connections.
5. Train the U-Net model on the Oxford Pet dataset for a fixed number of epochs.
6. Evaluate the trained model using accuracy and IoU (Intersection over Union) metric.
7. Visualize predictions: show original image, true mask, and predicted mask for sample test images.
8. Document training curves (loss/accuracy per epoch) and confusion metrics if applicable.

Starter Code (condensed)

Below is simplified starter code. Students must expand with full training, evaluation, and visualization.

```
import tensorflow as tf
from tensorflow.keras import layers, models
```

```
# Simple U-Net block
def unet_block(x, filters):
    c = layers.Conv2D(filters, (3,3), activation='relu', padding='same')(x)
    c = layers.Conv2D(filters, (3,3), activation='relu', padding='same')(c)
    return c

def build_unet(input_shape=(128,128,3), num_classes=1):
    inputs = layers.Input(shape=input_shape)
    # Encoder
    c1 = unet_block(inputs, 64); p1 = layers.MaxPooling2D((2,2))(c1)
    c2 = unet_block(p1, 128); p2 = layers.MaxPooling2D((2,2))(c2)
    # Bottleneck
    bn = unet_block(p2, 256)
    # Decoder
    u1 = layers.UpSampling2D((2,2))(bn)
    u1 = layers.Concatenate()([u1, c2])
    c3 = unet_block(u1, 128)
    u2 = layers.UpSampling2D((2,2))(c3)
    u2 = layers.Concatenate()([u2, c1])
    c4 = unet_block(u2, 64)
    outputs = layers.Conv2D(num_classes, (1,1), activation='sigmoid')(c4)
    return models.Model(inputs, outputs)

model = build_unet()
model.summary()
```

Deliverables

- Working code for single image U-Net demo (input vs predicted mask).
- Code implementation of U-Net model architecture.
- Training & validation plots for Oxford Pet dataset.
- Visual results: input image, ground truth mask, and predicted mask.
- Report containing explanation, screenshots, and observations.

Additional Challenges (Optional)

- Train U-Net with data augmentation (random flips, rotations).
- Implement IoU metric and compare with accuracy.
- Extend U-Net with dropout or batch normalization layers.
- Experiment with deeper U-Net or pretrained encoder (e.g., ResNet).